AI-vs-AI Tower Defense

Using A* Pathfinding, Minimax Decision-Making, and Intelligent Agents

Artificial Intelligence Project Report

Report by : Mrunal Jadhav RBT23CS039

Abstract

This project presents an AI-vs-AI Tower Defense simulation that integrates three fundamental artificial intelligence techniques: the A* search algorithm, the Minimax adversarial decision-making algorithm, and autonomous agent architecture. In this simulation, two independent AI agents compete on a grid-based map. The Attacker Agent employs A* pathfinding to compute the shortest collision-free route from its spawn point to the goal, dynamically replanning its trajectory whenever new obstacles are introduced. The Defender Agent applies the Minimax algorithm to evaluate candidate tower placements, selecting the position that maximally increases the attacker's path length or eliminates reachable routes entirely. Together, these agents create a fully autonomous zero-sum competitive environment that demonstrates core AI principles in a visual and interactive context. The system highlights how classical search algorithms and game-theoretic decision-making can be combined within an agent framework to produce intelligent, adaptive behaviour without human input.

1. Introduction

Tower defense games are a well-established genre in which a defending player places obstacles or weapons to prevent an attacking unit from traversing a map and reaching a goal. When both the attacker and defender are controlled by AI agents, the game becomes a self-contained testbed for adversarial artificial intelligence. This project implements exactly such a system, allowing two AI agents to compete entirely autonomously.

The three AI concepts used in this project are introduced below:

- **A* Algorithm:** A heuristic-based graph search algorithm that finds the optimal (shortest) path between two nodes. It combines the cost to reach a node (g-score) with an estimated cost to the goal (heuristic h-score), making it both complete and optimal for consistent heuristics.
- **Minimax Algorithm:** A recursive adversarial search algorithm used in two-player zero-sum games. The Maximiser (defender) attempts to find moves that maximise a score, while the Minimiser (attacker) attempts to minimise it. In this project, the score is the length of the attacker's A* path.
- **Intelligent Agents:** Autonomous entities that perceive their environment, reason about it, and take actions. Each agent in this project follows the standard Perceive-Decide-Act loop, operating independently without human intervention.

2. Objectives

The primary objectives of this project are:

1. Implement the A* pathfinding algorithm for the Attacker Agent to find optimal routes through a dynamic grid map.
2. Implement the Minimax algorithm for the Defender Agent to place towers at positions that maximally obstruct the attacker.
3. Design autonomous agents using the Perceive-Decide-Act loop so that both agents operate independently without human control.
4. Demonstrate adversarial AI by creating a zero-sum game where one agent's gain is the other's loss.
5. Visualise the interaction between both agents, showing A* path recalculation in real time as towers are placed.

3. Source Code

 # A* Pathfinding Algorithm (Attacker)

```
def heuristic(a, b):

    return abs(a[0] - b[0]) + abs(a[1] - b[1])



def astar(grid, start, goal):

    open_set = []

    heapq.heappush(open_set, (0, start))

    came_from = {}

    g_score = {start: 0}

    f_score = {start: heuristic(start, goal)}



    while open_set:

        _, current = heapq.heappop(open_set)

        if current == goal:

            path = []

            while current in came_from:

                path.append(current)

                current = came_from[current]

            return path[::-1]



        for dx, dy in [(-1,0),(1,0),(0,-1),(0,1)]:

            neighbor = (current[0]+dx, current[1]+dy)

            if not in_bounds(neighbor, grid): continue

            if grid[neighbor[0]][neighbor[1]] == 1: continue  # wall/tower
```

```
        tentative_g = g_score[current] + 1
        if tentative_g < g_score.get(neighbor, float('inf')):
            came_from[neighbor] = current
            g_score[neighbor] = tentative_g
            f_score[neighbor] = tentative_g + heuristic(neighbor, goal)
            heapq.heappush(open_set, (f_score[neighbor], neighbor))
    return None  # No path found
```

Minimax Decision Algorithm (Defender)

```
def minimax(grid, towers_placed, depth, is_defender, attacker_start, goal):
    path = astar(grid, attacker_start, goal)

    # Base case: max depth or terminal state
    if depth == 0 or path is None:
        return len(path) if path else float('inf'), None

    if is_defender:
        best_score = -float('inf')
        best_move = None
        # Defender tries placing a tower on each path cell
        for cell in path[1:-1]:
            grid[cell[0]][cell[1]] = 1  # Place tower
            score, _ = minimax(grid, towers_placed+1, depth-1, False, attacker_start, goal)
            grid[cell[0]][cell[1]] = 0  # Undo
            if score > best_score:
                best_score = score
                best_move = cell
        return best_score, best_move
    else:
        # Attacker minimizes path length via A*
        path = astar(grid, attacker_start, goal)
        return (len(path) if path else float('inf')), None
```

Agent Classes (Perceive-Decide-Act)

```
class AttackerAgent:

    def __init__(self, start, goal, grid):

        self.pos = start

        self.goal = goal

        self.grid = grid


    def perceive(self):

        return self.grid  # Full grid visibility


    def decide(self):

        path = astar(self.grid, self.pos, self.goal)

        return path[1] if path and len(path) > 1 else None


    def act(self):

        next_pos = self.decide()

        if next_pos:

            self.pos = next_pos

        return self.pos == self.goal  # True if reached goal




class DefenderAgent:

    def __init__(self, grid, goal):

        self.grid = grid

        self.goal = goal


    def perceive(self, attacker_pos):

        return astar(self.grid, attacker_pos, self.goal)


    def decide(self, attacker_pos):

        _, best_tower = minimax(self.grid, 0, depth=3,
```

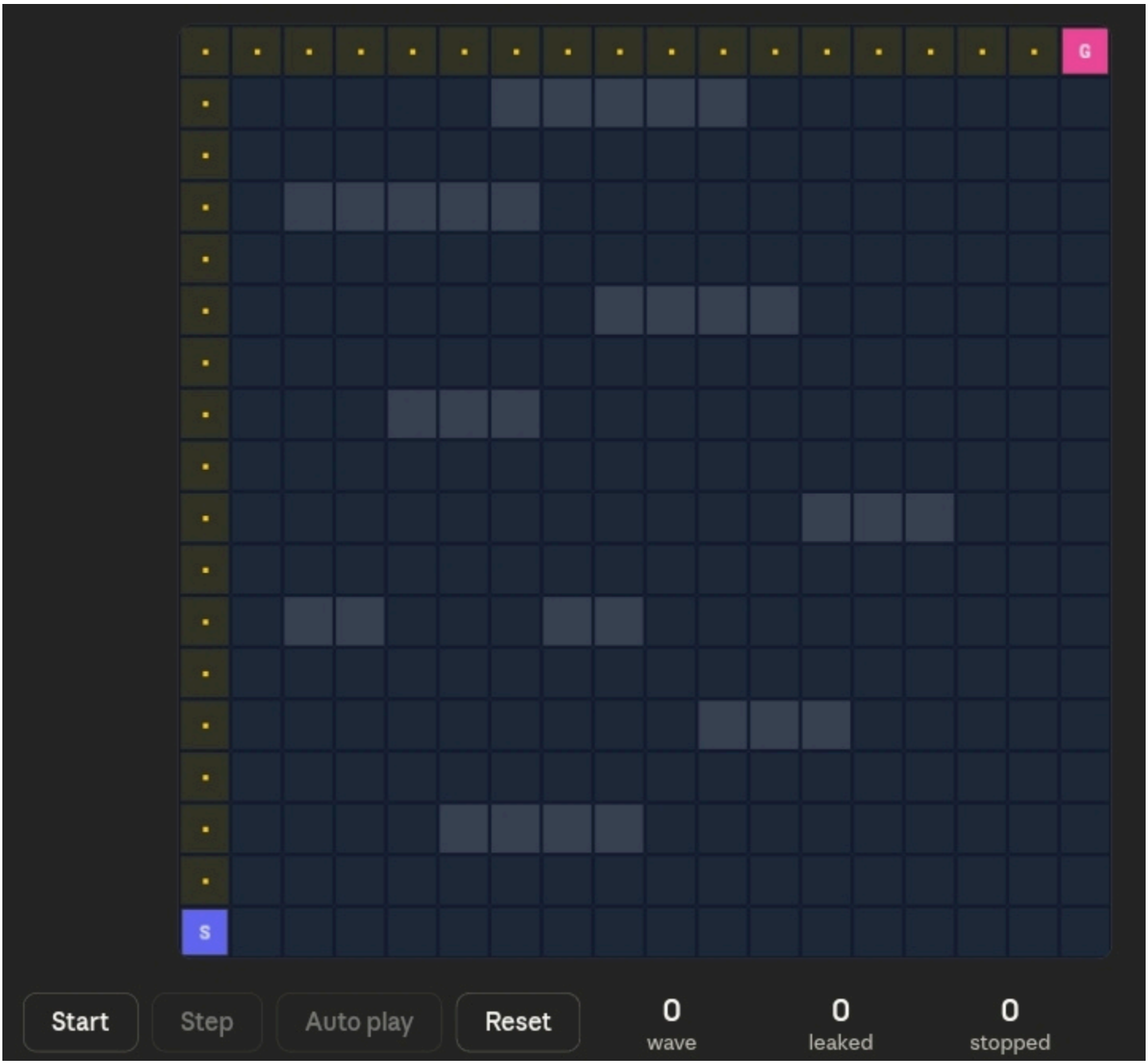
```
                                is_defender=True,
                                attacker_start=attacker_pos,
                                goal=self.goal)
        return best_tower

    def act(self, attacker_pos):
        pos = self.decide(attacker_pos)
        if pos:
            self.grid[pos[0]][pos[1]] = 1  # Place tower
```

Main Simulation Loop

```
import heapq
ROWS, COLS = 10, 10
grid = [[0]*COLS for _ in range(ROWS)]

# Add some walls
for c in range(3, 7): grid[3][c] = 1
for c in range(2, 6): grid[6][c] = 1

start = (9, 0)
goal  = (0, 9)

attacker = AttackerAgent(start, goal, grid)
defender = DefenderAgent(grid, goal)
# Phase 1: Defender places towers using Minimax
print("=== Defender Phase (Minimax) ===")
for i in range(4):
    tower = defender.act(attacker.pos)
    if tower:
        print(f"Tower {i+1} placed at {tower}")
# Phase 2: Attacker navigates using A*
print("\n=== Attacker Phase (A*) ===")
```

```
steps = 0

while True:

    reached = attacker.act()

    steps += 1

    print(f"Step {steps}: Attacker at {attacker.pos}")

    if reached:

        print("Attacker reached the goal!")

        break

    if steps > 100:

        print("Attacker is blocked — Defender wins!")

        break
```

4.output


G
S
Start
Step
Auto play
Reset
0
wave
0
leaked
0
stopped